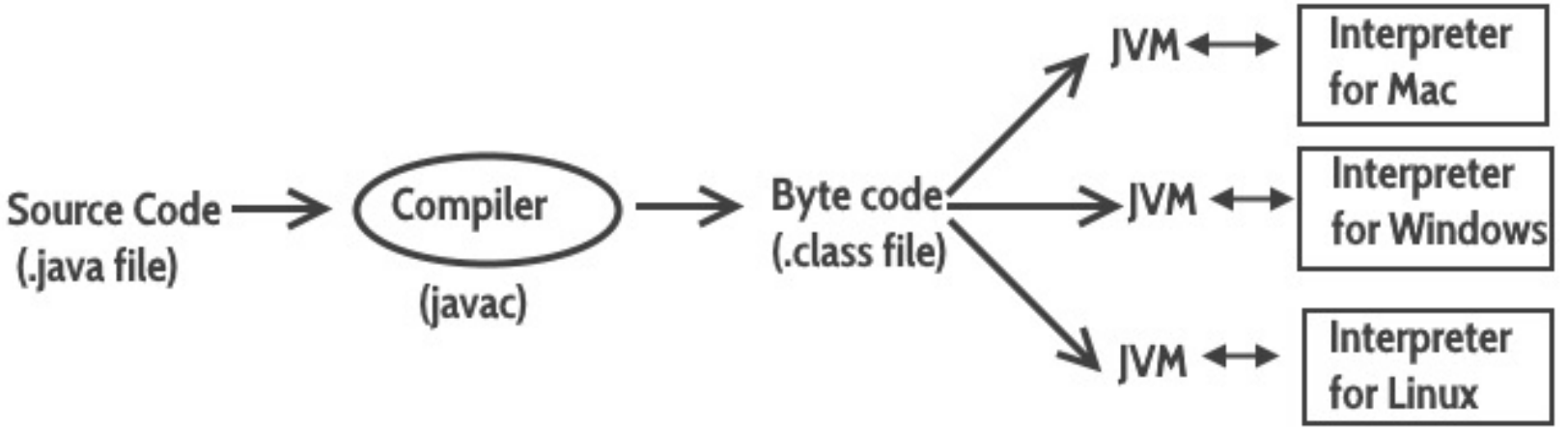
Object Oriented Programming

Java

What is JVM?

- Java Virtual Machine (JVM) is a part of the JDK and the foundation of the Java platform.
- It is a virtual machine that resides in the real machine and the **machine language for JVM is byte code.**
- JVM executes the bytecode generated by compiler and produce output. **JVM is the one that makes java platform independent**.



What is JIT?

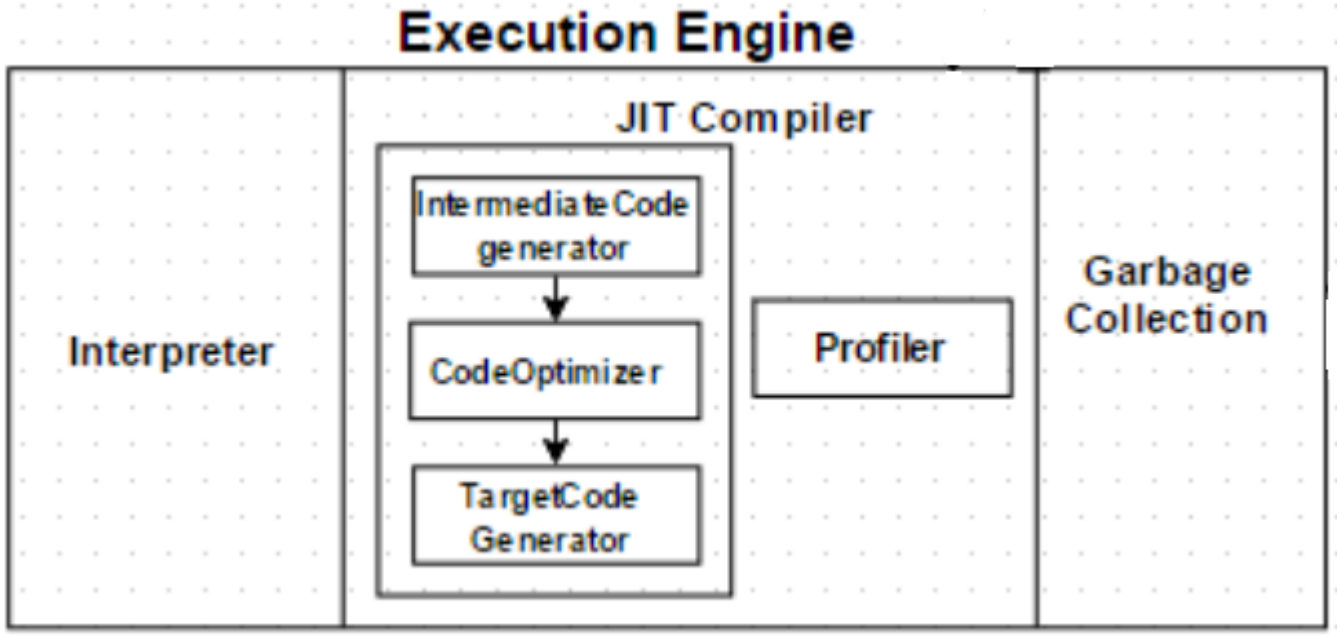
- JIT is part of JVM itself and used to improve performance of JVM. JIT stands for **Just In time** compilation.
- Bytecodes are then executed by JVM. Since execution of bytecode is **slower** than execution of machine language code, because JVM first needs to translate byte code into machine language code.
- JIT helps JVM here by compiling **currently** executing bytecode into machine language.
- JIT also offers **caching** of compiled code which result in improved performance of JVM.

Comparison Between Interpreter & Compiler

BASIS	COMPILER	INTERPRETER
Input	It takes an entire program at a time.	It takes a single line of code or instruction at a time.
Output	It generates intermediate object code.	It does not produce any intermediate object code.
Working mechanism	The compilation is done before execution.	Compilation and execution take place simultaneously.
Speed	Comparatively faster	Slower
Memory	Memory requirement is more due to the creation of object code.	It requires less memory as it does not create intermediate object code.
Errors	Display all errors after compilation, all at the same time.	Displays error of each line one by one.
Error detection	Difficult	Easier comparatively
Programming languages	C, C++, C#, Scala, typescript uses compiler.	Java, PHP, Perl, Python, Ruby uses an interpreter.

Execution Engine

- The Execution Engine will be using the help of the interpreter in converting bytecode, but when it finds repeated code it uses the JIT compiler, which compiles the entire bytecode and changes it to native code.
- This native code will be used directly for repeated method calls, which improve the performance of the system.



Class & Object in Java

- Java is a true OOP language and therefore the underlying structure of all Java programs is classes.
- Anything we wish to represent in Java must be encapsulated in a class that defines the “state” and “behaviour” of the basic program components known as objects.
- A class essentially serves as a template for an object and behaves like a basic data type “int”. It is therefore important to understand how the fields and methods are defined in a class.

Object-Oriented Programming Language

- Java supports the following fundamental concepts −
  - Classes
  - Objects
  - Instance
  - Method
  - Message Parsing
  - Polymorphism
  - Inheritance
  - Encapsulation
  - Abstraction

Object in Java

- An entity that has state and behavior is known as an object e.g. chair, bike, marker, pen, table, car etc.
- An object has three characteristics:
  - **state**: represents data (value) of an object.
  - **behavior**: represents the behavior (functionality) of an object such as deposit, withdraw etc.
  - **identity**: Object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. But, it is used internally by the JVM to identify each object uniquely.

Object in Java

- For Example: Pen is an object. Its name is Reynolds, color is white etc. known as its state. It is used to write, so writing is its behavior.
- **Object is an instance of a class**. Class is a template or blueprint from which objects are created. So object is the instance(result) of a class.
- **Examples**:
  - **Object**: Car
  - **State**: Color
  - **Behavior**: Climb Uphill, Accelerate, Slow Down

Class In Java

- A class is a group of objects that has common properties. It is a template or blueprint from which objects are created.
- A class in java can contain:
  - data member
  - method
  - constructor
  - block
  - class and interface
- A class in declared using **class** keyword in java.

Object & Class In Java

Java Naming Conventions

Name	Convention
class	should start with uppercase letter and be a noun e.g. String, Color, Button, System, Thread etc.
interface	should start with uppercase letter and be an adjective e.g. Runnable, Remote.
method	should start with lowercase letter and be a verb e.g. actionPerformed(), main(), print(), println() etc.
variable	should start with lowercase letter e.g. firstName, orderNumber etc.
package	should be in lowercase letter e.g. java, lang, sql, util etc.
constants	should be in uppercase letter. e.g. RED, YELLOW, MAX_PRIORITY etc.